KNIME Workflows in Scientific Studies: Publication Practices and Reproducibility Risks

Vitalii Kaplan[0009-0009-8181-2863]

2333275007@alu.istinye.edu.tr

Abstract. In this paper, we study how visual workflows are reported in scientific publications and how incomplete workflow preservation can make older studies harder to reproduce as workflow platforms evolve. We use KNIME as a case study, combining OpenAlex bibliometrics, a top-60 article audit registry, a manual compatibility test using current KNIME, and repository-level mining of KNIME node metadata. OpenAlex shows that KNIME is visible across a broad scientific literature, while the article assessment shows that influential studies often report KNIME through figures, descriptions, or tool mentions rather than downloadable workflow artifacts. In the top-60 audit registry, fifteen records report downloadable or linked KNIME workflow files. We could obtain workflow artifacts for four article records and open five KNIME workflows from those artifacts in a current local KNIME environment; only one workflow executed successfully. Repository mining gives the source-evolution context: deprecated ordinary nodes increased from 14.83% of registered ordinary nodes in the 2018 source baseline to 33.33% in the June 28, 2026 local source snapshot. Together, these results show that weak workflow preservation and evolving node metadata can make long-term reproduction of KNIME-based studies harder over time.

Keywords: Workflow reproducibility · KNIME · Deprecated nodes · Software evolution

1 Introduction

KNIME is a widely used open-source platform for constructing scientific data-analysis workflows, with more than a decade of use across published studies. Its visual workflows represent analyses as connected nodes with stored settings, data ports, and extension dependencies, which can make computational work easier to inspect, reuse, and repeat. This representation provides more operational detail than a narrative method description, but it does not remove the need to preserve the executable workflow artifact and its context. Reproduction requires access to the workflow file, KNIME version, extensions, data, and execution environment. Figures and textual summaries do not provide sufficient substitutes for these materials.

This issue matters for visual workflow platforms in general, because their reproducibility value depends on preserving executable workflow artifacts and the software context needed to interpret them. We study KNIME because it is an open-source visual workflow platform with a long public development history and substantial visibility in scientific literature. Our OpenAlex query returned 963 article-type records whose titles or abstracts match `KNIME`. The matching records span platform papers, extension papers, tool-comparison papers, and domain studies that use KNIME as an analysis interface. KNIME is therefore a suitable case for studying whether visual workflows are preserved as executable artifacts or only reported through descriptions and figures.

Our top-cited article collection now covers the 60 most-cited KNIME-matching records. The completed structured audit subset reported in Table 3 shows that publication practice is the weak link. Among the assessed non-KNIME-focused records, eight report downloadable or linked KNIME workflow files. Current retrieval and manual opening results were mixed: workflow artifacts were obtained for four article records, five KNIME workflows were opened, and only one workflow executed successfully in the tested local environment. For other influential studies, the strongest workflow evidence is usually weaker than an executable artifact. Several records present screenshots or figures of connected KNIME nodes, or describe workflow steps in text, without preserving all executable workflow context.

These reporting practices are risky because KNIME continues to evolve. A workflow that was executable when a paper was written may later depend on nodes that are deprecated, hidden, migrated, removed, or supplied by third-party extensions that are no longer easy to install. This paper therefore studies two connected forms of evidence: how KNIME workflows are reported in scientific articles, and how the KNIME source-code ecosystem changes around those workflows. We combine OpenAlex bibliometrics, top-cited article retrieval and assessment, current-version workflow opening and execution attempts, and repository-level mining of KNIME node metadata. The goal is to show why publication practice and source-code evolution must be considered together when evaluating long-term reproducibility of KNIME-based studies.

The rest of the paper is organized as follows. Section 2 reviews related work on computational reproducibility, open-source workflow systems, environment preservation, and platform evolution. Section 3 presents the methods before the data because the study depends on how the bibliometric records, article assessments, and repository snapshots were collected. Section 4 describes the resulting datasets. Section 5 reports the bibliometric, article-assessment, manual compatibility-test, and repository-mining results. Section 6 discusses the implications, limitations, and future extensions of the study.

2 Literature Review

Reproducibility is a central requirement for computational research because published results increasingly depend on software, data transformations, and execution environments. Provenance research makes this requirement explicit: visual analytics systems need records of data, analytical processes, and human interpretation to make later inspection and reuse possible [17]. Studies of executable research artifacts show the same problem in practice. Samuel and Mietchen found that biomedical Jupyter notebooks often fail during dependency installation or re-execution even when the notebooks themselves are available [14].

Open-source scientific software helps reproducibility by making algorithms, interfaces, and execution infrastructure inspectable. Weka, for example, is presented as an extensible open-source data-mining environment with graphical interfaces, reusable configurations, and plugin mechanisms [3]. KNIME is similarly used as an open-source visual workflow platform, and recent GIScience work explicitly proposes KNIME-based workflows as an approach for advancing replicable and reproducible research [2]. However, openness of the platform alone is insufficient. A workflow-based study also requires the executable workflow file, input data, platform version, installed extensions, and other software dependencies. Container research makes the same point at the environment level: scientific computations are easier to reproduce when complete software environments can be preserved and moved between systems [12]. More broadly, the testing literature shows that executable artifacts can remain sensitive to operating systems, dependency versions, timing, external services, and other environmental factors [16].

Even when an executable workflow is published, long-term reproduction can be affected by the evolution of the workflow platform itself. KNIME's node extension schema states that a deprecated node is no longer shown in the node repository, but is still loaded in existing workflows [10]. This design supports backward compatibility, while also marking a change in the ordinary authoring surface. KNIME workflows also persist node factory class names, and the platform provides extension points for mapping old factory classes to newer implementations and for registering workflow-node migration rules [9, 11]. These mechanisms show that continued workflow loading may depend on compatibility metadata, extension availability, and migration support. We therefore treat KNIME node deprecation as a measurable risk marker for long-term workflow reproducibility.

3 Methods

All scripts and data used for this study, except the other articles assessed as study objects, are available in the public replication repository [6]. Relative paths reported in this paper should be interpreted as paths within that GitHub repository.

3.1 Bibliometric Evidence

We collected bibliometric evidence from the OpenAlex Works API [13] on June 29, 2026. The query used OpenAlex title-and-abstract search for the word `KNIME` and restricted the result set to OpenAlex records whose type is `article`. The same query can be reproduced in the OpenAlex web interface by selecting "Search title and abstract", entering `KNIME`, and filtering the result type to `article`. The corresponding web-interface request can be represented as:

```
https://openalex.org/works
page=1
sort=relevance_score:desc
search.title_and_abstract=KNIME
filter=type:article
```

The first API request, which returns the first 200 records, is reproducible as:

```
https://api.openalex.org/works
?search.title_and_abstract=KNIME
&filter=type:article
&per-page=200
&cursor=*
```

OpenAlex uses cursor pagination for large result sets. After the first request, the collector reads `meta.next_cursor` from each response and submits the same request with `cursor=<next-cursor>` until no further results are returned. The value `per-page=200` is the maximum page size used here.

The collection script is `scripts/collect_openalex_knime_works.py`. It stores raw and processed collection artifacts as follows:

- raw OpenAlex response pages: `data/original/openalex/pages/`
- complete record stream: `data/original/openalex/works.jsonl`
- compact CSV view: `data/original/openalex/works.csv`
- endpoint, query parameters, collection time, result count, and page manifest: `data/original/openalex/metadata.json`

The current run collected 963 records across 5 OpenAlex response pages. Derived bibliometric tables are generated from the JSONL file by `scripts/build_openalex_knime_bibliometrics.py`.

3.2 Top-Cited Article Assessment

For manual full-text retrieval and assessment, we sorted the processed OpenAlex records by `cited_by_count` in descending order and retained the 60 most-cited records in `data/processed/openalex/openalex_knime_most_cited.csv`. We then attempted to retrieve all 60 articles. Open-access papers were downloaded directly, and non-open papers were requested through the author's institutional subscription access or added manually when available. At the time of this draft, 49 of the 60 records had local PDF files and processed one-column text. The structured audit retains all 60 records: downloaded papers receive detailed fields and quote-supported flags, while records without local full text are retained as not-assessed placeholders. For audited records, we manually inspected each retrieved paper, using LLM-assisted text analysis only to support extraction and consistency checking from the accessible full text. The assessment was organized as a structured audit rather than a free-form note file. First, each record was assigned descriptive fields for the article identifier, title, year, venue, DOI or URL, full-text access status, and the relation of the paper to KNIME. The KNIME relation separates papers about KNIME from papers that use KNIME in a domain study, tool comparison, interface, or implementation context. Papers classified as background or platform papers about KNIME were kept in the audited sample, but were not included in the deeper workflow-reporting flag audit.

Second, for papers that use KNIME, we recorded statistics-ready Boolean flags for the publication details relevant to reproducibility: whether the paper reports a KNIME version; provides downloadable KNIME workflow files; shows a KNIME workflow in figures or screenshots; describes workflow steps, nodes, or components in text; reports input-data availability; gives a direct data resource URL; provides code or scripts; reports KNIME extension or plugin dependencies; reports an extension installation source; and whether linked workflow artifacts were documented as retrievable in the project notes. The question guide used for these fields is stored in `data/processed/audit/knime_article_audit_questions.json`.

Third, each positive article-text flag was supported by a direct quote from the extracted article text. The quote-support object records the processed text line range, the quoted phrase or sentence block, the article section, and an audit note explaining the evidence. Candidate quotes were checked against the meaning of the corresponding question; if a candidate quote did not support the flag, another quote was sought. If repeated candidate quotes did not support the intended answer and no supporting quote was found, the flag was set to false. This rule makes a positive flag depend on article wording rather than on a keyword match alone.

The structured assessment is stored in `data/processed/audit/knime_most_cited_article_assessments.json`. Local article PDFs were converted to extracted text with `scripts/extract_article_texts.py`; where needed, two-column extracted text was normalized into reading order with `scripts/normalize_article_text_columns.py`. The support and provenance fields in the audit JSON were refreshed from the processed text with `scripts/refresh_article_audit_support.py`. This first pass is a document and artifact-availability audit. It does not execute workflows, verify every external URL, or test workflow import in KNIME.

3.3 Repository-Level Analysis

We study KNIME node deprecation through repository mining of the public `knime-oss` GitHub organization. KNIME Analytics Platform is a workflow-based data-analysis system in which analyses are assembled from connected processing nodes [1, 8]. KNIME extensions are distributed through update sites and extend the platform with additional functionality [7]. The KNIME source repositories were cloned locally, and each source snapshot was reconstructed by checking out every non-hidden top-level Git repository to the latest commit at or before a selected target date. This checkout step is implemented by `scripts/checkout_knime_oss_by_date.sh`. The script records a manifest for each snapshot, including repository name, checkout status, target date, selected commit, selected commit date, and previous head.

The current repository-mining corpus contains 91 immediate Git repositories in the local clone. The date-based mining process includes 90 non-hidden immediate repositories. The hidden `.github` repository is excluded because it contains organization metadata rather than KNIME node implementation code. The exact repository list is stored in `data/processed/knime_snapshots/knime_oss_repositories.csv`.

For each date-based source snapshot, we parse KNIME metadata files rather than grepping source text. The main deprecation marker is `deprecated="true"` on ordinary node registrations in `plugin.xml` under the Eclipse extension point

```
org.knime.workbench.repository.nodes
```

The KNIME schema states that deprecated nodes are no longer shown in the node repository but are still loaded in existing workflows [10]. Dynamic node sets use a separate extension point:

```
org.knime.workbench.repository.nodesets
```

We count these registrations separately. Node-description XML files ending in `NodeFactory.xml` and rooted at `knimeNode` are parsed as a secondary deprecation source. Hidden nodes, marked with `hidden="true"`, are counted separately from deprecated nodes.

The extraction also records compatibility-related metadata from two extension points:

```
org.knime.core.NodeFactoryClassMapper
```

```
org.knime.workflow.migration.NodeMigrationRule
```

Together, these extension points document mechanisms for mapping persisted node-factory class names to newer implementations and for registering workflow-node migration rules [9, 11]. These records are not counted as deprecation markers, but they are relevant for later analysis of whether deprecated or replaced nodes have explicit migration support.

The parser is implemented in `scripts/collect_knime_node_snapshot.py`. It uses Python's standard XML parser and skips `.git`, `target`, `bin`, and `.metadata` directories. Boolean XML attributes are treated as true only when their value is case-insensitively equal to `true`.

For each snapshot, the extractor writes per-record tables for node registrations, node descriptions, factory-class mappers, migration rules, and a local summary. The per-snapshot outputs are stored under `data/original/knime_snapshots/<snapshot-date>/` and include:

- `checkout_<snapshot-date>.csv`

- plugin_nodes.csv
- node_descriptions.csv
- factory_class_mappers.csv
- migration_rules.csv
- summary.csv

The cross-snapshot table is stored as:

```
data/processed/knime_snapshots/
knime_node_snapshot_summary.csv
```

It is generated from these per-snapshot outputs by `scripts/build_knime_node_snapshot_summary.py`.

The summary builder aggregates ordinary node registrations, dynamic nodesets, unique factory classes, deprecated nodes, hidden nodes, node-description files, deprecated node descriptions, factory-class mappers, and migration rules. It also counts processed and skipped repositories from each checkout manifest. Transition columns compare adjacent chronological snapshots. Node identity is based on `factory_class` where available. Otherwise, the fallback key is `plugin_xml:element:category_path`. These transition counts are therefore metadata-level approximations and are strongest for nodes with stable factory classes.

All project Python scripts are intended to be run with Python 3.11 or newer. The current scripts use only the Python standard library. No third-party Python packages are required for the OpenAlex and KNIME repository-mining pipelines.

4 Data

4.1 Bibliometric Evidence

The OpenAlex bibliometric data are organized into original API responses and processed analysis tables. The original data are stored under `data/original/openalex/`. They include raw response pages, a complete JSONL export, a compact CSV export, and query metadata. These files preserve the article-level records used to measure the size, time trend, venues, fields, and accessibility of the KNIME-matching literature.

Derived bibliometric tables are stored under `data/processed/openalex/`. Their roles in the analysis are:

- `openalex/openalex_knime_articles.csv`: compact article-level metadata used as the main processed bibliography table.
- `openalex/openalex_knime_articles_by_year.csv`: annual publication counts used to describe the time trend of KNIME-matching literature.
- `openalex/openalex_knime_top_venues.csv`: source-level aggregation used to identify journals, proceedings, repositories, and platforms where the matching records appear.

- `openalex/openalex_knime_top_domains.csv`: OpenAlex domain aggregation used to describe the broad disciplinary distribution.
- `openalex/openalex_knime_top_fields.csv`: OpenAlex field aggregation used to describe the finer disciplinary distribution.
- `openalex/openalex_knime_most_cited.csv`: citation-ranked subset used to select influential records for later manual assessment.
- `openalex/openalex_knime_likely_workflow_platform.csv`: heuristic subset used to prioritize papers likely to use KNIME as a workflow platform.

The processed OpenAlex directory also contains additional summary files:

- `openalex/openalex_knime_open_access_availability.json`
- `openalex/openalex_knime_role_classification_summary.json`

These files summarize artifact-access signals and role-classification counts.

4.2 Top-Cited Article Assessment

The top-cited article dataset records the manual assessment of the 20 citation-ranked OpenAlex records selected in Methods. The public replication package contains the processed assessment metadata, but not the article PDFs. The processed assessment is stored in `data/processed/articles/knime_most_cited_article_assessments.json`. The ranked source list is also available in human-readable form as `data/processed/openalex/most_cited.md`. Each record in this JSON file stores the OpenAlex rank, manual assessment status, KNIME role, reported KNIME version, workflow-artifact status, data and code availability, linked resources visible in the paper, and short evidence notes. Records without accessible full text are retained in the JSON with the status:

```
unable_to_download_or_not_present
```

This keeps the sample denominator explicit.

The PAINS filters paper is the only assessed top-cited applied-study case in which we found explicitly retrievable KNIME workflow files [15]. The assessment records the following reproduction-relevant identifiers:

- Article DOI: `https://doi.org/10.1002/minf.201100076`.
- KNIME version: `2.3.4`.
- RDKit nodes: `1.0.0.886`.
- Indigo nodes: `1.0.0.951`.
- myExperiment workflow: `http://www.myexperiment.org/workflows/1841`.
- myExperiment workflow: `http://www.myexperiment.org/workflows/2164`.
- Input-data reference: `http://blog.rguha.net/?p=850`.

These fields make the case useful for compatibility testing because they identify the workflow source, software versions, extension dependencies, and input-data context that must be checked before import or execution in a current KNIME release.

4.3 Repository-Level Analysis

The KNIME repository-mining data are organized as original per-snapshot audit files and one processed cross-snapshot summary. The processed summary is stored as:

`data/processed/knime_snapshots/`
`knime_node_snapshot_summary.csv`.

Per-snapshot audit files are stored under `data/original/knime_snapshots/<snapshot-date>/` and include:

- `checkout_<snapshot-date>.csv`
- `plugin_nodes.csv`
- `node_descriptions.csv`
- `factory_class_mappers.csv`
- `migration_rules.csv`
- `summary.csv`

Table 1: Columns in the processed KNIME node snapshot summary.

Name	Type	Short description
`snapshot_id`	string	Stable identifier for the snapshot.
`snapshot_kind`	categorical	Snapshot basis, currently date-based.
`snapshot_date`	date	Target date used for repository checkout.
`knime_version`	string	Version label or source-date context assigned by the summary script.
`source_basis`	categorical	Method used to obtain the source snapshot.
`repos_processed`	integer	Repositories with a checkout commit in the snapshot manifest.
`repos_skipped`	integer	Repositories without a commit at or before the target date.
`plugin_xml_files`	integer	Distinct parsed `plugin.xml` files contributing node metadata.
`repos_with_plugin_xml`	integer	Repositories contributing at least one parsed `plugin.xml`.

Name	Type	Short description
`registered_nodes`	integer	Ordinary node registrations in the node extension point.
`registered_nodesets`	integer	Dynamic nodeset registrations in the nodeset extension point.
`registered_total`	integer	Sum of ordinary node and nodeset registrations.
`unique_factory_classes`	integer	Distinct non-empty factory classes among ordinary node registrations.
`deprecated_nodes`	integer	Ordinary nodes with `deprecated="true"`.
`deprecated_nodesets`	integer	Nodesets with `deprecated="true"`.
`deprecated_total`	integer	Sum of deprecated ordinary nodes and deprecated nodesets.
`unique_deprecated _factory_classes`	integer	Distinct factory classes among deprecated ordinary nodes.
`deprecated_node_percent`	percent	Deprecated ordinary nodes divided by registered ordinary nodes.
`hidden_nodes`	integer	Ordinary nodes with `hidden="true"`.
`hidden_node_percent`	percent	Hidden ordinary nodes divided by registered ordinary nodes.
`deprecated_and _hidden_nodes`	integer	Ordinary nodes marked both deprecated and hidden.
`node_description_files`	integer	Parsed `*NodeFactory.xml` files rooted at `knimeNode`.
`description_deprecated _files`	integer	Node-description files with `deprecated="true"`.
`description_deprecated _percent`	percent	Deprecated node descriptions divided by parsed node descriptions.
`factory_class_mapper _count`	integer	Registered `NodeFactoryClassMapper` contributions.
`migration_rule_count`	integer	Registered `NodeMigrationRule` contributions.
`nodes_added _since_previous`	integer/blank	Node identity keys present in this snapshot but not the previous one.

Name	Type	Short description
nodes_removed _since_previous	integer/blank	Node identity keys present in the previous snapshot but absent here.
nodes_newly_deprecated _since_previous	integer/blank	Common node keys that became deprecated since the previous snapshot.
nodes_no_longer_deprecated _since_previous	integer/blank	Common node keys no longer marked deprecated.
nodes_newly_hidden _since_previous	integer/blank	Common node keys that became hidden since the previous snapshot.
nodes_no_longer_hidden _since_previous	integer/blank	Common node keys no longer marked hidden.
nodes_category_changed _since_previous	integer/blank	Common node keys whose observed category-path set changed.

5 Results

5.1 Bibliometric Evidence

The OpenAlex title-and-abstract query returned 963 article-type records. This is the first piece of evidence for the study: KNIME is visible in a sizeable scientific literature, so preservation of KNIME workflows is not only a niche platform-maintenance issue. The annual series indicates that KNIME-matching literature becomes visible in OpenAlex from the mid-2000s and grows into a sustained publication stream after 2015 (Fig. 1). The highest annual count in the current export is 104 records in 2023. Counts for 2026 are incomplete because the query was collected on June 29, 2026.

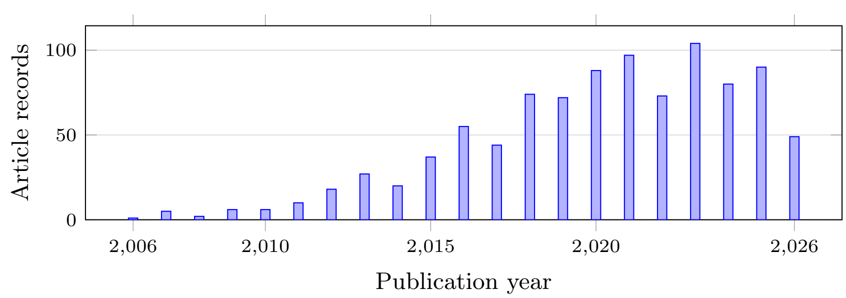

Fig. 1. OpenAlex article records matching KNIME in title or abstract by publication year.

The domain distribution shows that the KNIME-matching literature is not confined to one disciplinary area (Table 2). OpenAlex assigns just over half of the records to Physical Sciences, but the remaining records are spread across Social Sciences, Life Sciences, and Health Sciences. This matters for the study because workflow reproducibility risk is not only a software-engineering concern. If KNIME workflows are used across computational chemistry, biomedical analysis, education, business analytics, and related areas, then missing workflow artifacts and changing node implementations can affect published computational evidence in several research communities.

Table 2. OpenAlex domains for KNIME title-and-abstract matches.

Domain	Articles	Share
Physical Sciences	505	52.44%
Social Sciences	187	19.42%
Life Sciences	149	15.47%
Health Sciences	122	12.67%

The ten largest OpenAlex fields are led by Computer Science (350), Biochemistry, Genetics and Molecular Biology (115), Medicine (92), Engineering (71), and Social Sciences (66). These field counts show why a paper about KNIME reproducibility cannot focus only on the KNIME platform literature. The central reproducibility question is how scientific studies that use KNIME as a tool report the executable workflow, data, versions, and extensions needed for later reproduction.

Open-access metadata indicate 619 open-access records, 625 records with some full-text availability, and 421 records with a PDF signal. These signals are important because later workflow audits require access to paper text, supplements, repositories, and sometimes PDF-only workflow descriptions. The counts are only availability indicators. They do not by themselves establish that a workflow artifact, KNIME version, input data, or extension list is available.

The heuristic role classification separates likely papers about KNIME or KNIME extensions from papers that likely use KNIME as an analysis tool. The current rules classify 156 records as about KNIME or a KNIME extension, 314 as using KNIME as a tool, 28 as about or mentioning KNIME in the title, and 465 as ambiguous OpenAlex matches. The same heuristic marks 410 records as likely using KNIME as a workflow platform. These labels are useful for triage: they identify a substantial candidate set where workflow reporting can matter, but final case-study claims require manual full-text and artifact assessment.

5.2 Top-Cited Article Assessment

We built a top-60 retrieval pool from the most-cited OpenAlex records matching `KNIME`. The statistics in this subsection use the top-60 structured audit registry, converted into explicit article-level flags. The purpose of this part of the study is not to decide whether KNIME is visible in the literature; the bibliometric data already show that. The purpose is to ask whether highly cited KNIME-matching papers preserve the evidence needed to understand or reproduce a KNIME-based workflow: full text, KNIME-use role, version information, workflow files, workflow figures or textual descriptions, data, code, and extension context.

The audit separates background and platform papers from deeper workflow-reporting cases. Six of the 60 records are about KNIME itself or a KNIME extension. These records are important for understanding the KNIME ecosystem, but they do not need the same artifact-preservation audit as domain studies that use KNIME. We therefore retain their descriptive metadata but leave their workflow-reporting flag maps empty. Table 3 uses all 60 top-cited records as the denominator. It first reports the classification of the records, then reports the workflow-reporting flags observed in the structured audit.

Table 3 summarizes the statistics-ready flags in the structured audit file. Thirty-six records use KNIME as a workflow, tool, interface, or implementation context. Thirty records contain textual workflow or node descriptions, and nineteen show workflow screenshots or figures. Fifteen records report downloadable or linked KNIME workflow files. Current retrieval attempts have so far obtained workflow artifacts for four of these article records; one additional repository was reachable but did not contain a KNIME workflow file. Version and dependency reporting remain limited: eleven records report a KNIME version, eleven report KNIME extension or plugin dependencies, and six report an installation source. Data and code signals are more common than retrievable workflow-artifact evidence: twenty-nine records report input-data availability, fourteen provide a direct input-data resource, and ten provide code or scripts. This pattern is important because a reader may find data or code without being able to recover the actual KNIME workflow, node settings, KNIME version, or extension context.

Table 3: Flag summary for the top-cited article audit. Shares use all 60 top-cited records as the denominator.

Audit category or flag	Count	Share of 60	Interpretation
All top-cited records	60	100.0%	Records retained from the OpenAlex ranking
Assessed from full text	49	81.7%	Article text available for manual assessment
Not assessed from full text	11	18.3%	Full text unavailable at assessment time
About KNIME	6	10.0%	Background/platform or extension papers, not deeper workflow-reporting cases
Not a KNIME use case	7	11.7%	KNIME appears in the record but is not a KNIME-based study
Uses KNIME	36	60.0%	KNIME used as a workflow, tool, interface, or implementation context

Audit category or flag	Count	Share of 60	Interpretation
Workflow or nodes described in text	30	50.0%	Named workflow steps, nodes, modules, or components recorded
Workflow screenshots or figures	19	31.7%	Workflow shown visually in article or supplement
Reports KNIME version	11	18.3%	Specific KNIME version value found
Downloadable KNIME workflow files	15	25.0%	Executable workflow files provided or linked
Reports extension/plugin dependencies	11	18.3%	KNIME extension or node-library dependency reported
Reports extension installation source	6	10.0%	Update site, project URL, or installation source reported
Linked workflow artifacts retrieved	4	6.7%	Linked KNIME workflow artifacts obtained in project notes
Provides code or scripts	10	16.7%	Code repository, scripts, or software code reported
Reports input-data availability	29	48.3%	Data availability stated, with or without direct URL
Direct input-data resource	14	23.3%	Direct data URL or resource pointer recorded

Table 4: Workflow-reporting signals among the 36 assessed KNIME-use cases.

Audit flag	Count	Share of KNIME-use cases
Uses KNIME	36	100.0%
Workflow or nodes described in text	30	83.3%
Workflow screenshots or figures	19	52.8%
Reports KNIME version	11	30.6%
Downloadable KNIME workflow files	15	41.7%
Reports extension/plugin dependencies	11	30.6%
Reports extension installation source	6	16.7%
Linked workflow artifacts retrieved	4	11.1%

The workflow-level experiment then tested the eight earlier article records that reported downloadable or linked KNIME workflow files before the rank 41–60 audit expansion. The purpose was limited: we checked whether the linked artifacts could be obtained, whether a current local KNIME installation could open them, and whether execution completed without manual repair. Four article records yielded workflow artifacts. Because the PAINS record contains separate RDKit and Indigo workflows, these four article records produced five workflow-opening attempts. Only the PAINS RDKit workflow executed successfully. The other opened workflows either failed during execution, required missing R packages or extensions, or could not be confidently executed from the available workflow state. Across the opened workflows, the screenshots also show deprecated or legacy nodes, but we treat this as qualitative compatibility evidence here rather than counting visible deprecated nodes. The screenshots are archived under each workflow directory's `opened/` subdirectory in `data/original/workflows/`.

Table 5: Manual workflow retrieval, opening, and execution results for the eight article records with downloadable or linked KNIME workflow files.

Article record or workflow	Retrieval result	Manual KNIME result	Screenshot evidence
PAINS RDKit, DOI `10.1002/minf.201100076`	Manual workflow ZIP obtained after automated myExperiment download returned HTTP 403.	Opened and executed successfully in the current local KNIME environment.	Available
PAINS Indigo, DOI `10.1002/minf.201100076`	Manual workflow ZIP obtained after automated myExperiment download returned HTTP 403.	Opened, but execution was blocked by a missing Indigo KNIME Integration node and related data-cell implementation.	Available
Chemical data curation workflow, DOI `10.1186/s13321-018-0315-6`	GitHub archive obtained; KNIME workflow files found.	Opened, but execution failed inside a metanode during the manual test.	Available
ASD genes prediction workflow, DOI `10.1371/journal.pone.0208626`	GitHub archive obtained; inner KNIME workflow archive extracted.	Opened, but execution failed in an R node because required R packages and functions were unavailable in the test environment.	Available
Medicinal chemistry filters workflow, DOI `10.3390/encyclopedia3020035`	GitLab archive obtained; KNIME workflow files found.	Opened, but the manual test did not establish a successful end-to-end run; the available screenshots show unresolved or not-yet-configured workflow parts.	Available
BitterSweetForest workflow link, DOI `10.3389/fchem.2018.00093`	Not obtained. The original project page redirected to VirtualTaste, where the target page returned HTTP 404.	No KNIME opening test possible.	–
MS-Ready workflow link, DOI `10.1186/s13321-018-0299-2`	Repository archive obtained, but no `.knwf` or `workflow.knime` file was found in the extracted archive.	No KNIME opening test possible.	–
KNIME-OMICS workflow link, DOI `10.1016/j.jprot.2016.08.002`	Not obtained. The referenced GitHub organization URL returned HTTP 404.	No KNIME opening test possible.	–
Alzheimer ligand workflow link, DOI `10.1080/07391102.2018.1456975`	Not obtained. The referenced project site failed DNS resolution during the retrieval attempt.	No KNIME opening test possible.	–

5.3 Repository-Level Analysis

The repository-mining results explain why weak workflow reporting becomes more costly over time. Deprecated nodes are not an isolated metadata corner case in the KNIME source tree. At the earliest available source-code baseline, 2018-04-03, 193 of 1301 registered ordinary nodes were marked as deprecated, or 14.83% (Table 6). By the KNIME Analytics Platform 5.0.0 source-date anchor on 2023-02-22, the count had increased to 433 of 1442 ordinary nodes, or 30.03%. The current local source-date snapshot on June 28, 2026, contains 502 deprecated ordinary nodes out of 1506, or 33.33%. These figures show that the deprecated-node share approximately doubled between the 2018 baseline and the 2026 local snapshot.

The annual checkpoint table gives the same trend at regular intervals (Table 7). The number of repositories with checkout commits rises from 44 in 2018 to 90 in 2026, so early and late totals should not be compared as if the mined source corpus were fixed. Nevertheless, the deprecated-node share rises from 14.83% in 2018 to 33.47% at the 2026 annual checkpoint. Hidden ordinary nodes remain much less frequent than deprecated ordinary nodes, but they increase from 2 in 2018 to 24 in 2026. Deprecated dynamic nodesets appear from 2020 onward and remain at 2 in the annual checkpoints through 2026.

Table 6. Selected KNIME repository-mining snapshots.

Date	Version context	Nodes	Deprecated nodes	Deprecated share
2018-04-03	KNIME Analytics Platform 3.5.3 source-date anchor	1301	193	14.83%
2019-12-05	KNIME Analytics Platform 4.1.0 source-date anchor	1191	227	19.06%
2023-02-22	KNIME Analytics Platform 5.0.0 source-date anchor	1442	433	30.03%
2026-03-03	KNIME Analytics Platform 5.11.0 source-date anchor	1503	503	33.47%
2026-06-28	Current local source-date snapshot	1506	502	33.33%

The transition table indicates that node deprecation changes occur in bursts rather than at a constant rate (Table 8). The largest annual increases in newly deprecated node identities occur at the 2021, 2023, and 2026 checkpoints, with 80, 61, and 60 newly deprecated node keys, respectively. The 2022 checkpoint is the only annual checkpoint in which a substantial number of common node keys are no longer marked deprecated: 29. Category-path changes are also concentrated in a few years, especially 2021 and 2023. Because these transition counts use metadata-level node identity keys, they should be read as evidence of source metadata evolution, not as direct evidence that published workflows fail to open or execute.

These repository-level results directly address the second research question: how KNIME node deprecation evolved over time. They also provide necessary

Table 7. Annual KNIME repository-mining checkpoint statistics.

Year	Snapshot date	Repositories	Nodes	Deprecated nodes	Deprecated nodesets	Hidden nodes	Deprecated share
2018	2018-04-03	44	1301	193	0	2	14.83%
2019	2019-01-01	45	1500	248	0	2	16.53%
2020	2020-01-01	54	1200	228	2	2	19.00%
2021	2021-01-01	60	1366	398	2	5	29.14%
2022	2022-01-01	67	1424	386	2	8	27.11%
2023	2023-01-01	72	1428	433	2	10	30.32%
2024	2024-01-01	80	1434	436	2	9	30.40%
2025	2025-01-01	86	1488	446	2	23	29.97%
2026	2026-01-01	90	1509	505	2	24	33.47%

Table 8. Annual KNIME repository-mining transition statistics.

Year	Added	Removed	Newly deprecated	No longer deprecated	Category changed
2018	–	–	–	–	–
2019	194	22	16	0	6
2020	11	0	0	0	4
2021	140	1	80	0	111
2022	69	6	17	29	17
2023	56	4	61	0	67
2024	17	7	4	0	3
2025	36	0	9	0	1
2026	52	2	60	0	3

context for the workflow-level questions, because a published workflow that depends on older node identities may encounter deprecated, hidden, removed, or migrated components in a later KNIME version. However, the tables do not answer how often uploaded or executed workflows use deprecated nodes. That question requires workflow-level corpora and execution traces beyond repository metadata.

Taken together, the three evidence streams give the intended picture of the study. First, KNIME is visible in a broad and sustained scientific literature. Second, influential papers that use or mention KNIME rarely provide the full workflow-preservation package needed for later reproduction: executable workflow files, KNIME versions, extension context, data, and code. In the assessed non-KNIME-focused set, PAINS was the only case where we could retrieve KNIME workflow files and perform a current-version import experiment. Third, KNIME's source metadata continues to evolve, and roughly one third of ordinary registered nodes are marked deprecated in recent source snapshots. The result is not a claim that all KNIME workflows fail. The result is a reproducibility risk: as KNIME evolves, papers that preserve only workflow images or textual descriptions become harder to diagnose, migrate, and reproduce.

6 Discussion

To our knowledge, this is the first focused empirical study connecting KNIME workflow reporting in scientific publications with KNIME source-code evolution.

The contribution is intentionally narrower than a full reproducibility benchmark. We do not estimate the failure rate of all published KNIME workflows. Instead, we show a reproducibility risk pattern: KNIME is used in a visible scientific literature, influential papers often do not publish executable workflow artifacts, the one retrievable workflow case in our non-KNIME-focused assessment still depends on extension availability, and the KNIME node ecosystem has accumulated a substantial deprecated-node surface.

The results suggest three practical implications. First, a published workflow image is not an executable artifact. Several highly cited papers provide figures or textual descriptions of connected KNIME nodes, but this is not enough to recover node settings, extension versions, input paths, or migration requirements. Second, reporting a KNIME version is useful but incomplete. Workflows may also depend on third-party extension update sites and node libraries, as shown by the PAINS case where the RDKit workflow could be updated and executed while the Indigo workflow was blocked by a missing extension. Third, repository-level deprecation is a useful warning signal. It does not prove that a workflow fails, but it identifies a compatibility surface that grows as the platform evolves. Without careful workflow preservation and version reporting, this growth makes old workflows harder to inspect and repair.

The next development of this study should move from paper-level evidence to workflow-level evidence. One direction is to create a larger corpus of public KNIME workflows from supplements, myExperiment, KNIME Hub, GitHub, and other repositories, then test whether they open in current KNIME versions and which deprecated, hidden, migrated, or missing nodes they contain. A second direction is to link deprecated nodes to migration metadata and replacement nodes in the KNIME source tree, so that workflow warnings can distinguish low-risk deprecations from nodes that require unavailable extensions or manual repair. A third direction is to preserve reproducibility environments for selected case studies by recording operating system, KNIME version, installed extensions, update sites, input data, import warnings, execution errors, and output differences.

Tools and services outside the present evidence base can provide the next layer of usage evidence. `knime2py` can help identify KNIME workflows that users try to translate or operationalize outside KNIME [5], and `k2pweb.org` can provide anonymized aggregate evidence about real uploaded or executed workflows [4]. These sources would address a question that the current paper deliberately leaves open: how often researchers and analysts encounter deprecated KNIME nodes in practice, not only how often deprecated nodes appear in the source repository or in selected published artifacts.

References

1. Berthold, M.R., Cebron, N., Dill, F., Gabriel, T.R., Kötter, T., Meinl, T., Ohl, P., Sieb, C., Thiel, K., Wiswedel, B.: KNIME: The Konstanz information miner. In: Data Analysis, Machine Learning and Applications, pp. 319–326. Springer (2008). https://doi.org/10.1007/978-3-540-78246-9_38

2. Fu, X., Liu, L., Guan, W.W., Kalra, Y., Bao, S., Kötter, T., Sturm, K.: Advancing replicable and reproducible GIScience: an approach with KNIME. Cartography and Geographic Information Science **53**(3), 270–290 (2025). https://doi.org/10.1080/15230406.2024.2446556
3. Hall, M., Frank, E., Holmes, G., Pfahringer, B., Reutemann, P., Witten, I.H.: The WEKA data mining software: An update. ACM SIGKDD Explorations Newsletter **11**(1), 10–18 (2009). https://doi.org/10.1145/1656274.1656278
4. Kaplan, V.: k2p-web: KNIME to Python/Jupyter converter. `https://k2pweb.org/` (2026), accessed July 1, 2026
5. Kaplan, V.: knime2py: KNIME to Python workbook. `https://github.com/vitalii-kaplan/knime2py` (2026), accessed July 1, 2026
6. Kaplan, V.: reproducibility-risks-article: Replication repository for KNIME workflow reproducibility-risk study. `https://github.com/vitalii-kaplan/reproducibility-risks-article` (2026), accessed July 1, 2026
7. KNIME AG: Installing extensions and integrations. `https://docs.knime.com/ap/latest/analytics_platform_user_guide/index.html\#installing-extensions-and-integrations` (2026), accessed June 29, 2026
8. KNIME AG: KNIME Analytics Platform User Guide. `https://docs.knime.com/ap/latest/analytics_platform_user_guide/index.html` (2026), accessed June 29, 2026
9. KNIME AG: KNIME Core NodeFactoryClassMapper Extension Point Schema. `https://github.com/knime-oss/knime-core/blob/3885b31a49d047401a076981f9bd4c964b4a97a7/org.knime.core/schema/NodeFactoryClassMapper.exsd` (2026), accessed June 29, 2026
10. KNIME AG: KNIME Workbench Repository Node Extension Point Schema. `https://github.com/knime-oss/knime-workbench/blob/c94203746f98ab38cccea54e3497d62c9adee9e1/org.knime.workbench.repository/schema/Node.exsd` (2026), accessed June 29, 2026
11. KNIME AG: KNIME Workflow Migration NodeMigrationRule Extension Point Schema. `https://github.com/knime-oss/knime-database/blob/c4a3356985edf4383ad0a3107658851e6c49fdad/org.knime.workflow.migration/schema/NodeMigrationRule.exsd` (2026), accessed June 29, 2026
12. Kurtzer, G.M., Sochat, V., Bauer, M.W.: Singularity: Scientific containers for mobility of compute. PLOS ONE **12**(5), e0177459 (2017). https://doi.org/10.1371/journal.pone.0177459
13. Priem, J., Piwowar, H., Orr, R.: OpenAlex: A fully-open index of scholarly works, authors, venues, institutions, and concepts. arXiv preprint arXiv:2205.01833 (2022). https://doi.org/10.48550/arXiv.2205.01833, `https://arxiv.org/abs/2205.01833`
14. Samuel, S., Mietchen, D.: Computational reproducibility of Jupyter notebooks from biomedical publications. GigaScience **13** (2024). https://doi.org/10.1093/gigascience/giad113
15. Saubern, S., Guha, R., Baell, J.B.: KNIME workflow to assess PAINS filters in SMARTS format: Comparison of RDKit and Indigo cheminformatics libraries. Molecular Informatics **30**(10), 847–850 (2011). https://doi.org/10.1002/minf.201100076
16. Tahir, A., Rasheed, S., Dietrich, J., Hashemi, N., Zhang, L.: Test flakiness' causes, detection, impact and responses: A multivocal review. Journal of Systems and Software **206**, 111837 (2023). https://doi.org/10.1016/j.jss.2023.111837

17. Walker, R., Slingsby, A., Dykes, J., Xu, K., Wood, J., Nguyen, P.H., Stephens, D., Wong, B.L.W., Zheng, Y.: An extensible framework for provenance in human terrain visual analytics. IEEE Transactions on Visualization and Computer Graphics **19**(12), 2139–2148 (2013). https://doi.org/10.1109/TVCG.2013.132